

THE MIDNIGHT GHOST IN CLUSTER 9

Budget-Aware Causal Root-Cause Analysis for Microservices

Daksh Rathi | IIT Mandi

GitHub: github.com/dakshrathi-india/The-Midnight-Ghost-in-Cluster-9 | Live demo: cluster9-incident-rca.streamlit.app

Abstract

A failure in one microservice often shows up as alarms in several others. This project builds an incident investigator that tries to separate those symptoms from the actual root cause while working under a fixed telemetry-query budget. It combines robust metric detectors, semantic log evidence, trace-derived dependencies and explicit causal hypotheses. Once a diagnosis is resolved, remediation is safety-gated and recovery is checked again using fresh telemetry. Synthetic incidents are used because they give hidden ground truth for evaluation; the RCA itself never receives that truth. The same core pipeline can also consume canonical events produced from OpenTelemetry-style data.

1. Problem and scope

The main difficulty is not finding an abnormal service. In a cascade, many services can be abnormal at the same time. The harder question is which service and failure mechanism can explain the rest of the affected region. The system therefore treats RCA as a sequence of evidence-gathering and elimination steps rather than a single anomaly score.

- Incident queries are budgeted; the agent cannot read everything.
- Service clocks may be skewed and telemetry may arrive late or be missing.
- Service names and the dependency graph are not hard-coded into generic RCA logic.
- A remediation command being accepted is not treated as proof that the incident is solved.

2. Architecture

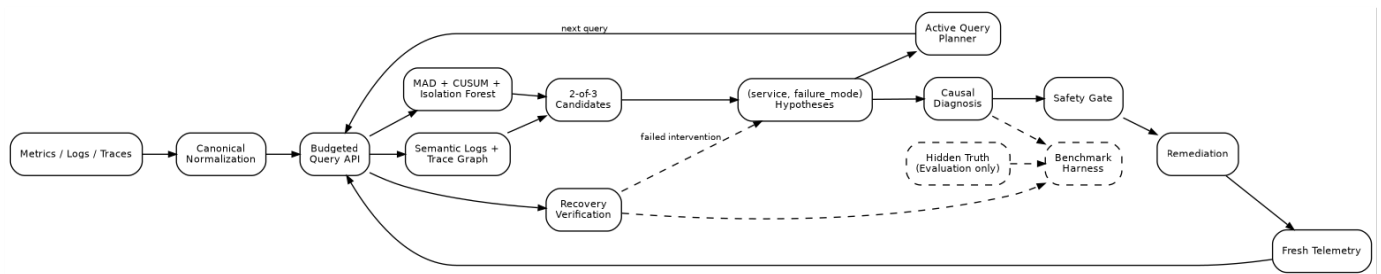


Figure 1. End-to-end flow used by the current implementation.

Raw metrics, logs and spans are normalized first. Historical healthy data provides baselines and cached dependency context. During an incident, all telemetry access goes through TelemetryQueryAPI so query cost and cache hits are visible. Candidate services are generated from metric detectors, then logs, traces and graph structure are used to evaluate explicit (service, failure_mode) hypotheses. If the diagnosis is strong enough, an allowlisted action can be attempted and a separate verification phase checks whether the system actually recovered.

Part	What it does
Telemetry + baseline	Canonical events preserve service, units and event/arrival time; healthy history supplies baselines and cached topology.
Budgeted queries	Every incident query is charged; exact repeats can be served from cache at zero extra cost.
RCA	Builds candidates, tests failure signatures, ranks hypotheses and asks for more evidence only when it can help.
Action + evaluation	Known actions are bounded and recovery is checked separately; hidden truth is joined only after inference for scoring.

3. How the diagnosis is built

Three metric detectors are kept separate because they catch different behaviour. Their outputs are evidence, not the final answer.

Signal	What it catches	Why it helps
MAD / modified-z	Large robust deviations from a healthy baseline	Less sensitive to a few baseline outliers than mean/std based rules.
CUSUM	Smaller but sustained directional changes	Useful when a fault builds over several samples instead of producing one spike.
Isolation Forest	Unusual combinations of core metrics	Adds a multivariate view when no single metric is extreme enough on its own.

Logs and traces are kept outside this detector vote. They are used as corroborating and causal evidence later, so a semantic match or a slow span cannot by itself create a metric candidate.

3.1 Candidate rule

For each service/window, 2 or 3 metric detectors firing gives STRONG, exactly 1 gives WEAK, and a single detector can be promoted to STRONG only when a failing trace or a high-severity semantic log corroborates it. If no metric detector fires, the service is not a candidate. This rule is intentionally simple enough to inspect during a failure.

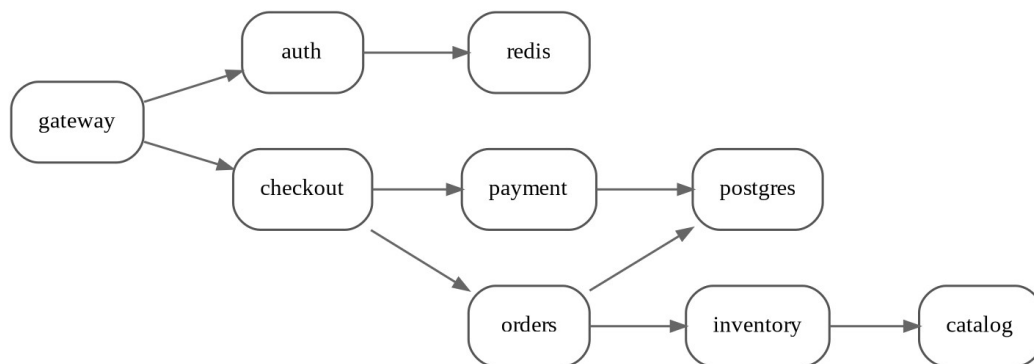


Figure 2. Synthetic benchmark topology. Generic RCA uses observed service names and dependency edges rather than special cases for these names.

3.2 Hypotheses, contradictions and active queries

Every candidate service is paired with the supported failure signatures. The evaluator records SUPPORT, CONTRADICTION or NEUTRAL evidence from metric direction, logs, traces, temporal order and graph propagation. Missing telemetry stays neutral; it is not treated as proof against a hypothesis.

Hypotheses are ordered lexicographically: fewer contradictions first, then fewer unexplained STRONG candidates, more explained STRONG candidates, more supporting evidence categories, and finally more direct support. If the causal key is still tied, the result stays AMBIGUOUS. The query planner then spends the remaining budget on an affordable log/trace/metric query that can separate the surviving alternatives.

4. Remediation and recovery

The action layer is intentionally narrower than the diagnosis layer. Only known failure modes have allowlisted actions, and the target remains a single diagnosed service. The current mapping is:

Failure mode	Action
cpu_saturation	SCALE_UP
deployment_regression	ROLLBACK
database_slowdown	FAILOVER
connection_exhaustion	RECYCLE_CONNECTIONS
network_latency	REROUTE
process_crash	RESTART

APPLIED only means the executor accepted the command. Recovery is evaluated separately from fresh telemetry. The verifier checks the diagnosed root, the affected region, request-rate continuity and a healthy sentinel. It can return VERIFIED, FAILED or INCONCLUSIVE. A decisive failed intervention is also fed back as a contradiction against the exact attempted hypothesis, so the next investigation does not blindly repeat the same explanation.

5. Simulation vs. real telemetry

The simulator is a test source, not part of the RCA reasoning. It creates reproducible incidents and keeps the injected root cause hidden so the result can be scored later. In a real deployment, the left side of the pipeline changes: an OpenTelemetry collector/export or another telemetry backend supplies the data, which is converted into the same canonical events.

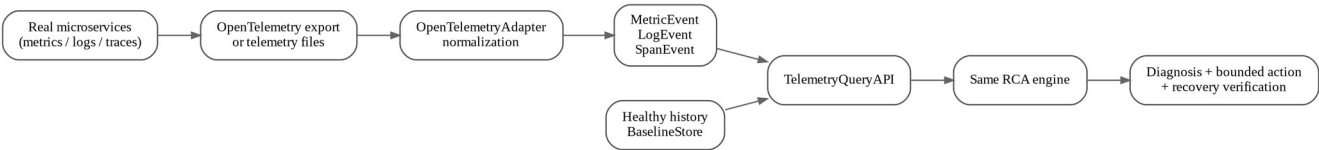


Figure 3. Production-facing path. The repository currently provides OTLP-style JSON/dict ingestion; a live collector receiver is not implemented yet.

This separation matters because the UI may ask which synthetic fault to inject, but those labels are not passed to the DiagnosisAgent. The agent sees only the resulting telemetry and historical baseline.

6. Evaluation

The benchmark uses valid service/failure-mode pairs derived from SimulationConfig. The full matrix below used 45 valid pairs across two seeds (90 incidents per profile). CLEAN is useful as a sanity check; DEFAULT is more realistic because it includes ordinary fragmentation and decoys.

Profile	Incidents	Exact pair	Resolved	Verified	False resolutions	Mean cost
CLEAN	90	98.9%	98.9%	61.1%	0	14.52
DEFAULT	90	66.7%	77.8%	34.4%	10	14.59

The important result is not the 98.9% clean number by itself. Under DEFAULT fragmentation, 10 diagnoses were false resolutions. None of the validated runs incorrectly marked a wrong action as VERIFIED, which is why recovery verification is kept separate from diagnosis accuracy.

6.1 Robustness subset

Profile	Exact	Resolved	Verified	False res.	Mean cost
CLEAN	100%	100%	57.1%	0	15.14
DEFAULT	85.7%	85.7%	57.1%	0	14.71
CLOCK_SKEW	71.4%	85.7%	28.6%	1	16.00
MISSING	100%	100%	57.1%	0	15.14
DELAYED	100%	100%	57.1%	0	15.14
NOISY	100%	100%	0%	0	10.14
DECOY	100%	100%	57.1%	0	15.14
COMBINED_STRESS	57.1%	71.4%	0%	1	10.86

Clock skew and combined stress hurt diagnosis most in this small subset. Noise is different: diagnosis stayed correct, but the verifier often refused to claim recovery. That conservative failure mode is preferable to reporting success without enough evidence.

6.2 Ablation and query budget

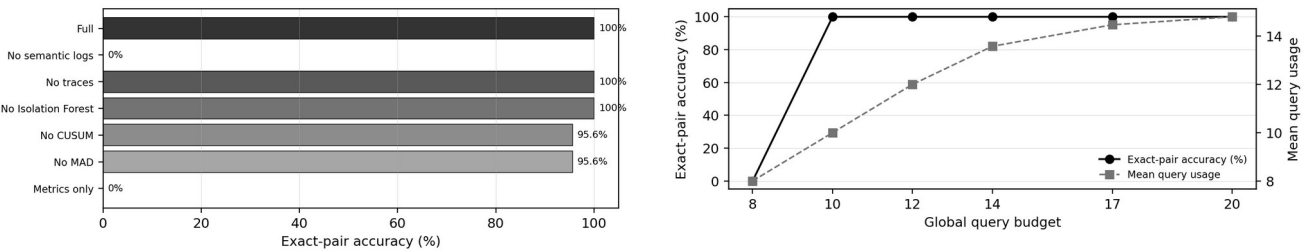


Figure 4. Left: CLEAN ablation study. Right: budget curve on the same controlled subset.

Semantic-log removal collapses resolution under the current conservative policy, while removing traces or Isolation Forest did not change exact accuracy on this particular clean subset. That should not be read as proof that those components are generally unnecessary. The budget curve shows a practical threshold: budget 8 is insufficient, while larger budgets stop being consumed once the remaining queries are no longer useful.

7. Current scope and limitations

The project is intentionally narrower than a production incident platform. The table below is the clearest way to state what is already implemented and what is still outside the validated checkpoint.

Implemented now	Not claimed yet
Single-root synthetic incidents with six supported failure modes	General multi-root or previously unseen failure mechanisms
OTLP-style JSON/dict adapter into canonical telemetry	A live OpenTelemetry collector receiver or long-running production ingestion service
Bounded simulator remediation executor and recovery checks	Production credentials, authorization and real infrastructure executors
Controlled robustness, ablation and budget experiments	Production accuracy or incident-frequency claims
Service/topology-agnostic generic RCA code	Guaranteed diagnosis under arbitrary missing/noisy telemetry

The main weakness visible in the benchmark is false resolution under fragmented DEFAULT telemetry. The final hardening adds open-set abstention for unsupported failure modes, a stricter orthogonal-evidence gate before autonomous action, healthy/no-fault controls, deterministic winner-vs-runner-up explanations, an isolated RCAEval adapter/smoke-test path, and explicit incident state history. These changes are covered by the final regression suite; the external adapter is not presented as a production-accuracy result.

8. Reproducibility

The final regression suite passes 195 tests. The reference run uses seed 17 with postgres/database_slowdown under the DEFAULT profile. It resolves to postgres/database_slowdown, plans FAILOVER, applies the action, verifies recovery and consumes 16 of 17 global query-cost units. The repository also contains deterministic benchmark commands, saved result formats, an OpenTelemetry adapter, external-evaluation adapters, and the Streamlit investigation UI.

9. Conclusion

The project is built around one practical idea: an abnormal service is not automatically the cause. The system first narrows the search with robust detectors, then uses logs, traces, topology and contradictions to test causal explanations under a fixed budget. Actions are bounded and recovery is checked independently. The simulator makes the evaluation repeatable, while the canonical telemetry layer keeps the RCA core usable beyond the simulator.